

SRA $\rightarrow$ PARu
MBRA $\rightarrow$ SIMRu

Rational Agent

- $\rightarrow$ makes informed decisions like a person, firm, s/w, machine
- $\rightarrow$ carries out the best outcome based on its past & current percepts.

AI System = Agent + Environment $\leftarrow$ contains mult. agents
 $\hookrightarrow$ acts on the envirn.

Agent:

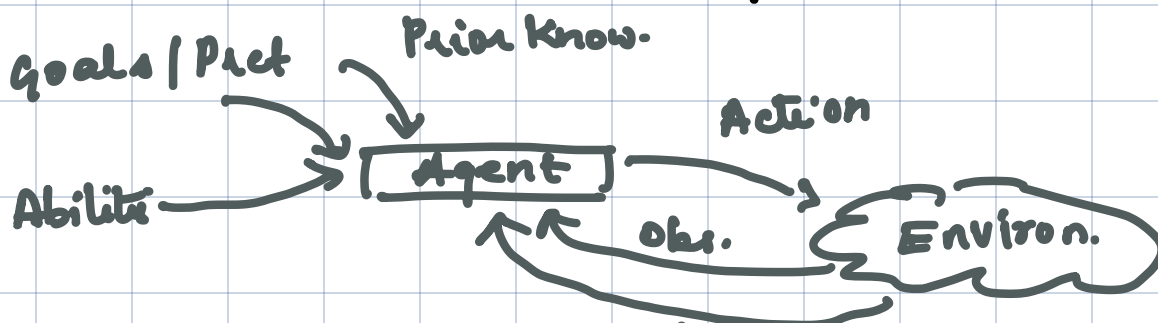
- perceives the environment through sensors
- acts upon " " " " actuators

ARCHITECTURE = SENSOR + ACTUATOR

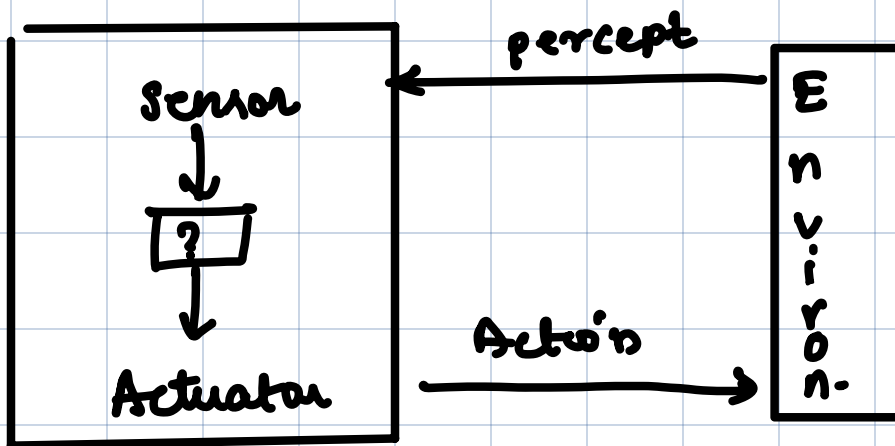
AGENT = ARCH. + PROGRAM

eg: Human agent $\rightarrow$ sensor: Eyes, nose, ears etc

Actuators: muscle, legs, voice



past exp.



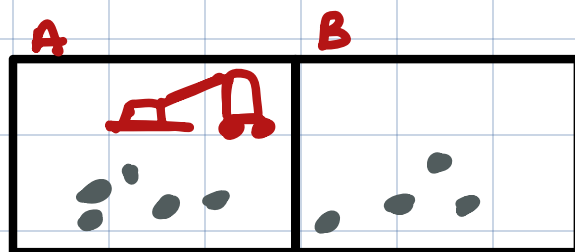
① Percept: a perceptual i/p at any instant

- Agent's PERCEPT SEQ. = complete history of everything perceived.
- Agent's choice of action maybe $\rightarrow$ Per. Seq. but not anything it hasn't perceived.

② Agent function: maps any given percept seq to action

③ Agent program: the code that implements the agent function for an AI agent.

VACUUM CLEANER PROBLEM



Problem Formulation

Initial state $\rightarrow$ state where the problem starts
A/B any state could be initial

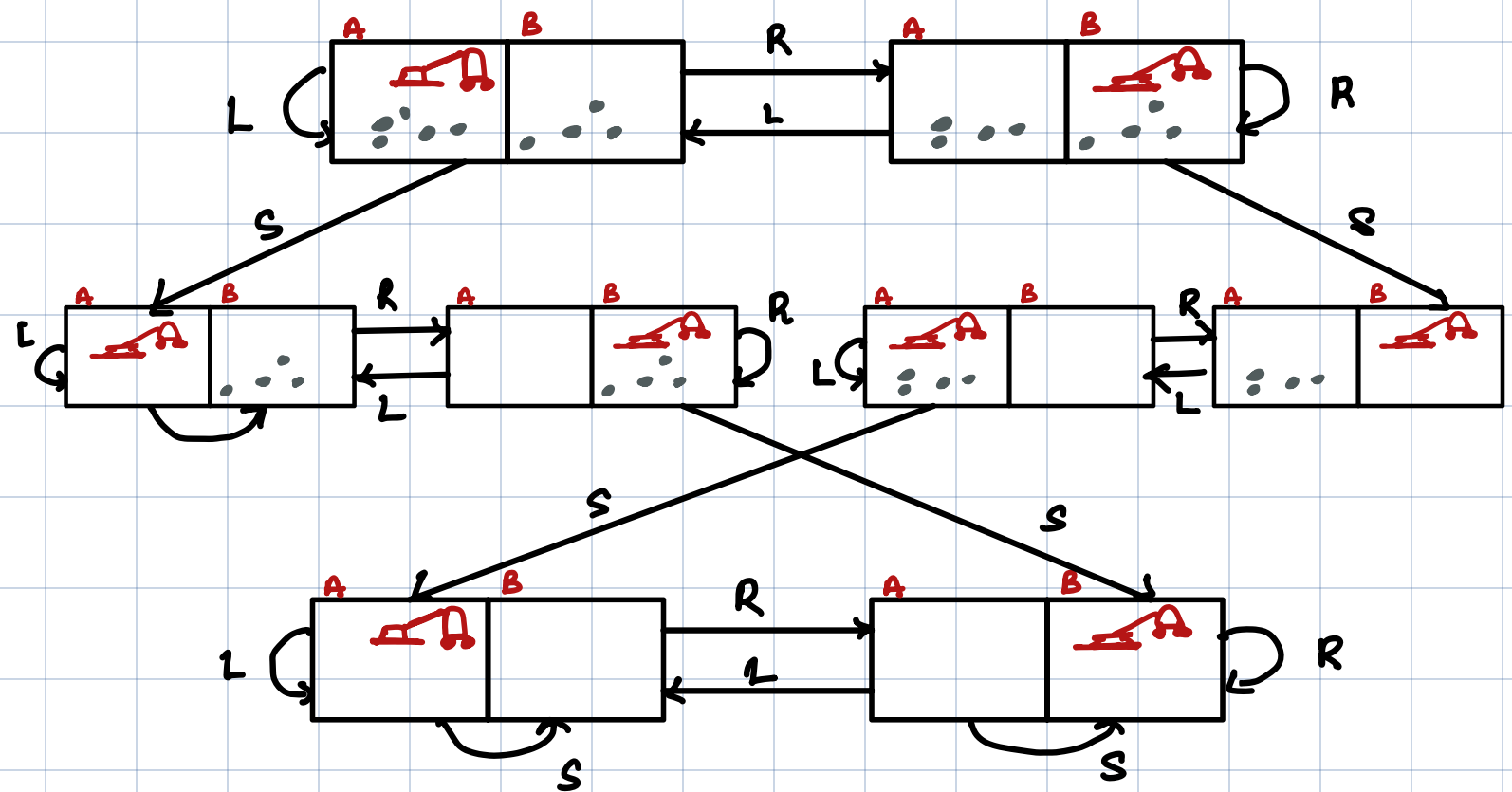
Actions $\rightarrow$ what are the actions agent could take
Move Left, Move Right, Suck

function $\rightarrow$ Result (s^{state} , a)
 $\sim$ action

Goal State $\rightarrow$ Both rooms A/B are clean

Path cost $\rightarrow$ Each step costs = 1
 $\therefore$ path cost = no. of steps

State Space / Transition



PEAS

- ① P: Both rooms clean, time taken, min energy consumption
- ② E: Two rooms (Clean / Dirty)
- ③ A: Motor for moving, suction tube to suck dirt, wheels
- ④ S: dirt sensor, location sensor, bumpers

Rationality of an agent depends on:

- ① Performance Measure → define the criteria for success.
- ② Agent's prior knowledge of environment
- ③ The action that it can perform
- ④ The agent's percept seq. upto date.

Properties it should have

- Gather info
- learn from the exp
- autonomy
- augm. knowledge

PEAS → type of model on which AI agents work.

- ① Performance Measure : Criteria to judge the success / perf. of the agent
- ② Environment: the real environment where the agent needs to deliberate its action.
- ③ Actuators : tools / equipments that agents use to perform actions on the env.
O/p of the agent

- ④ Sensors: tools / equipments that capture the state or perceive the environment.
I/p to the agent

Example

P → what qualities (qualities / client)
E → what is its surrounding
A → how will it perform action S → sense

System

PEAS

Auto-door OS

efficiency
P : time taken, power consumed, noise
E : Area on both sides of the door
A : Motors closing & opening " "
S : motion detection, camera on both sides

Face Auth System

P : accuracy, time taken, ab. to handle new
E : face detected by camera
A : authentication response
S : camera / sensor sensing presence

Automated Car
Driver

P : efficiency in navigation, safety, optimum speed
E : Roads, traffic, weather
A : steering wheel, AB, gears

S : cameras , lidar , object detection

Task Environment $\rightarrow$ refers to the actions, outcomes & choices a given user has for a given task.

* diff b/w task env & env
 $\nearrow$ world struct for the spec. task.
 $\nwarrow$ world

Properties: $\nearrow$ ⑦

- $\rightarrow$ Fully v/s Partially Observable
- $\rightarrow$ Single v/s Multi Agent
- $\rightarrow$ Deterministic v/s Stochastic
- $\rightarrow$ Episodic v/s Sequential
- $\rightarrow$ Static v/s Dynamic
- $\rightarrow$ Discrete v/s Continuous
- $\rightarrow$ known v/s unknown.

①

Fully Observable - agent's sensors give access to the complete state of the env. at each point in time.

↳ easy because no need for internal state to keep track history of the world.

Unobservable - agent has no sensors at all to the environment

* Env = P.O $\Rightarrow$ noise + inaccurate sensors / parts of state are simply missing

Eg:

FO $\rightarrow$ Chess (Board full visible + moves of opp)

PO $\rightarrow$ Driving (Blind corners + weather)

Poker (Opp. cards not known)

Angry Birds - FO

Minecraft / Temple Run - PO

Deterministic - next state of env completely depends on current state + action of the agent

↳ In practical situations → it is impossible to keep track of all aspects of a state ie unobserved aspects ⇒ STOCHASTIC

Eg: Driving → Stochastic
Vacuum → Deterministic

↙ simpler because no need to think ahead
Episodic - agent's exp. is divided into atomic episodes

↳ each ep = a single action
ie no part + no future effects

eg: Defect detection on part on assembly line

Sequential - current decision may affect all future decisions.

eg: Chess, Driving

Static v/s Dynamic



environment can change while an agent is deliberating $\Rightarrow$ Football

easy cause while taking a decision need not check the world or worry abt time.

$\Downarrow$
mowing lawn

Discrete v/s Continuous

$\hookrightarrow$ applies to 3 parts

- ① state of env.
- ② way time is handled
- ③ action + percept

Chess: $\rightarrow$ state: FINITE set of distinct states

time: exclude the clock

action + percept: set of discrete steps

Driving: continuous state + continuous time

Known → outcomes for actions are given
Unknown → agent learns how to work in
order to make good decisions
↓
man move

Single Agent — only one agent involved in
env & operating by itself.

Multi-Agent — multiple agents.

→ chess
driving partially cooperative

Kinds of Agents:

- ① Simple Reflex
- ② Model-Based Reflex
- ③ Goal Based
- ④ Utility
- ⑤ Learning Agents

Table Driven Agent

```
function TABLE_DRIVEN (percept)
    persistent: percepta (sequence)
    append percept to percepta
    ACTION → LOOKUP (percepta, table)
    return action.
```

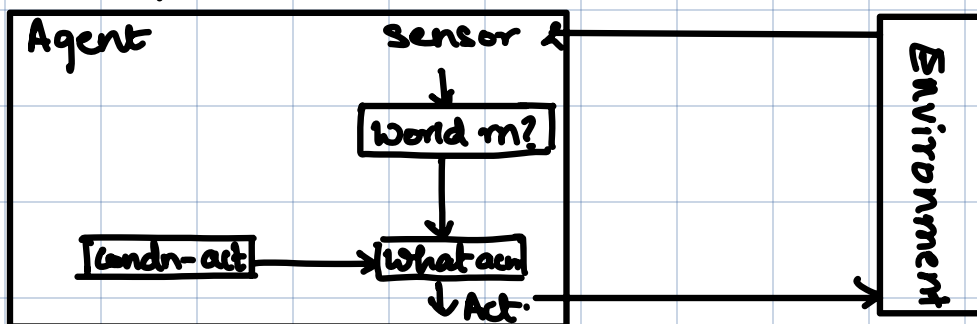
* look up table

has actions mapped
to certain seq.

Simple Reflex Agent

- Simplest kind of agent
- Agent choose action based on CURRENT percept & ignores percept history
- CONDITION - ACTION Rule
if [cond.] then [action]
- Fully Observable

Diagram



funcn. SRA (percept) returns action

persistent: rules, set condn-action rules
state ← INTERPRET (percept) *function generating an abstracted desc. of current state from percept*
rules ← RULE-MATCH (rules, state) *returns rules that match state*
action ← rules.ACTION
return action

Problem:

- ① Very limited intelligence
- ② No knowledge of non-percept. parts of state
- ③ Change in environment $\rightarrow$ rules $\rightarrow$ update

Eg:

Automatic Doors

Percept: Motion Detected

Action: Open Door

Rule: IF motion detected
 $\rightarrow$ open door

Thermostat

Percept: Current temp.

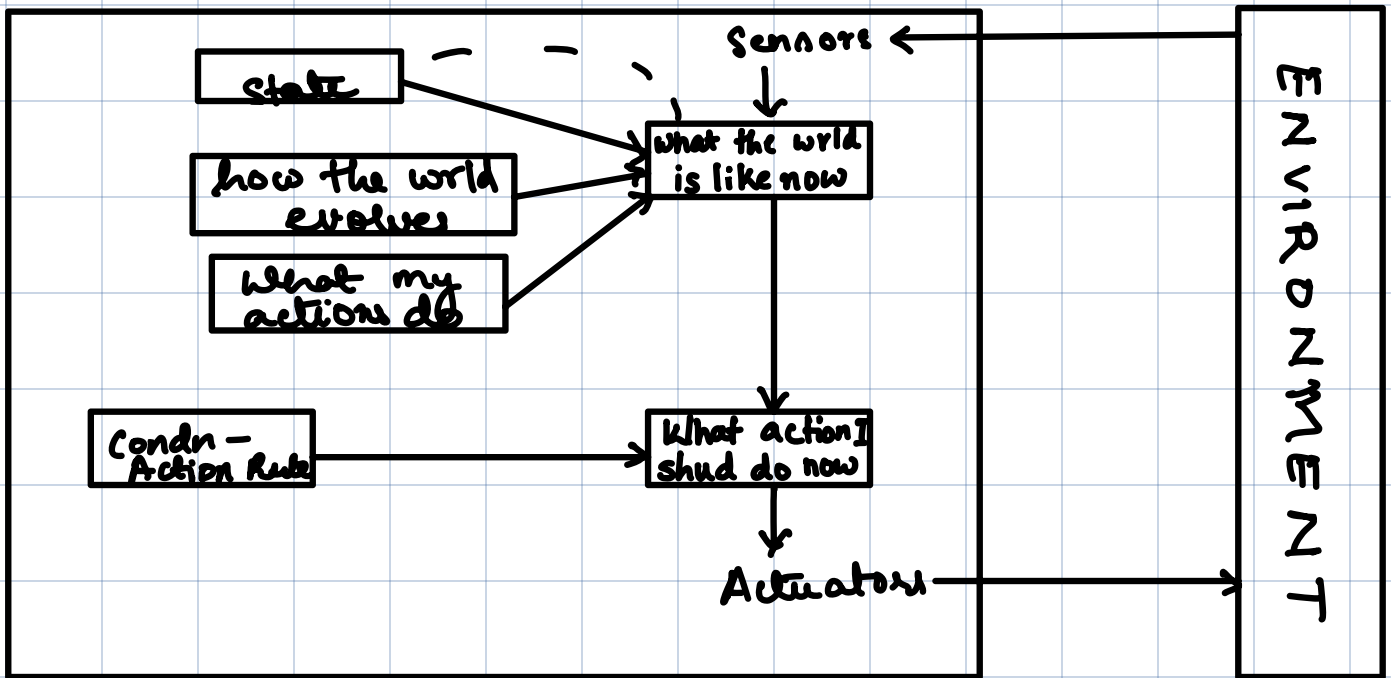
Action: Heater ON/OFF

Rule IF temp $< 20^{\circ}\text{C} \rightarrow$
ON

Model - Reflexive Agent

- $\rightarrow$ effective way to handle the partial observability, agent keeps track of the world it can't see.
- $\rightarrow$ Agent keep some internal state (depends on percept history thereby reflects at some unobserved aspects of the current state)

Perceive → Remember → Reason → Act



function MBRA (percept) returns action:

persistent:

state → the agent's perception of world state

model → how world evolves + action conseq.

rules → set of condn-action rules

action → the most recent action

new internal description
state ← Update - state (state, model, action, percept)

rules ← Rule - Match (state, rules)

action ← rules.ACTION

return action

Model = internal memory of the environment

stores:

- ① what the agent has seen before
- ② How world changes over time
- ③ How action affects env.

Updating Internal state info →

- ① How the world evolves independently of agent
- ② How the agent's own action affect the world

Eg: Robot Vacuum

Components:

1. Sensor

Dirt

Proximity

Location

Battery

2. Internal State

Map of rooms

Cleaned v/s uncleaned

Obstacle of path

Battery status

3. World Model

move fwd → posn change

batter ↓ → dock

area clean → no appear

obstacle → can't move

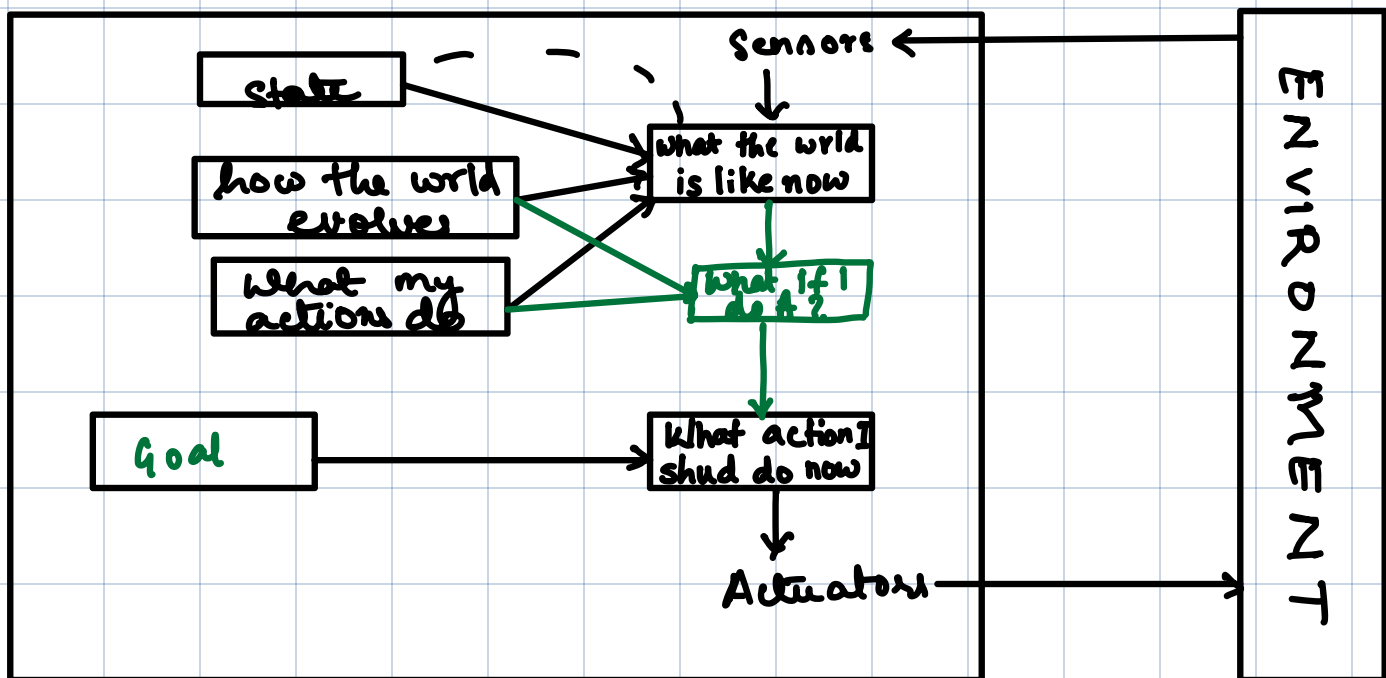
robot remembers & believes

affect of action
↑ how world works

4. Condition Action

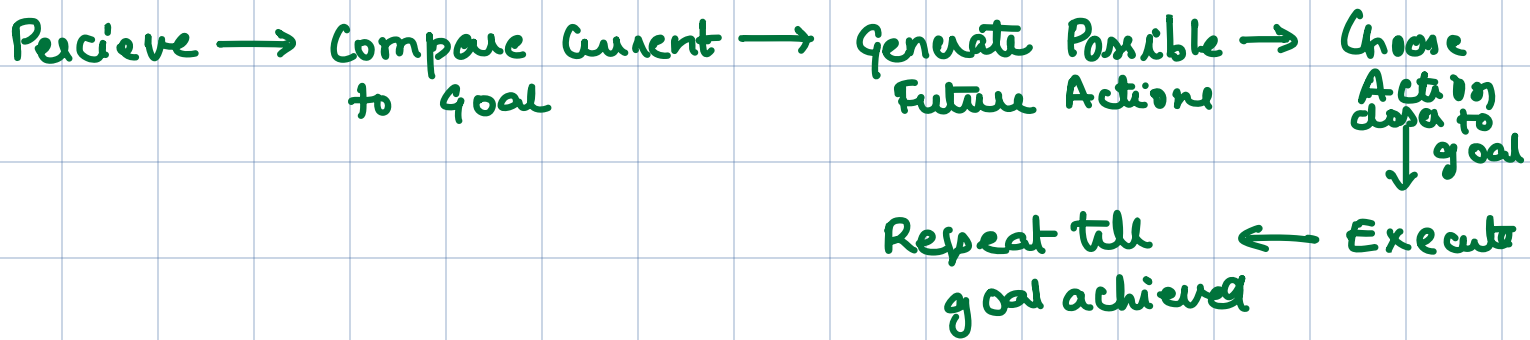
Area dirty	clean
Obstacle ahead	move
Battery low < 20	dock
Room cleaned	move to next room

Goal Based Agent



mode based goal agent keeps track of world state as well as a set of goals it trying to achieve

Knowing the current state of the environment is not always enough to decide what to do $\Rightarrow$ Goal
eg: Junction



Drone - Based Delivery Agent

Sensors:

GPS

Altitude

Obstacle (LiDAR)

Wind

Battery

State:

Current location: Warehouse

Target: Customer's house

Battery: 90%

Weather: Normal

Compare:

Current $\neq$ Goal

$\rightarrow$ Act

Generate Action

Fly $\downarrow$ alt.

Fly $\uparrow$ bldgs

check no fly zone

Planning & Search

Predict path.

estimate energy

Route A $\rightarrow$

Route B $\rightarrow$

Select Action

Execute

Re-Plan
(Dynamic)

Goal Ach.

Utility - Based Agents

Goals provide binary distinction b/w happy or unhappy state

Eg: Taxi can reach goal

- a) Fast + Risky route
- b) Medium & Cheap
- c) Safe + slow

} All are valid for GBA

→ Models & keeps track of the env. , tasks that have research based on - perception, sup, reasoning learning

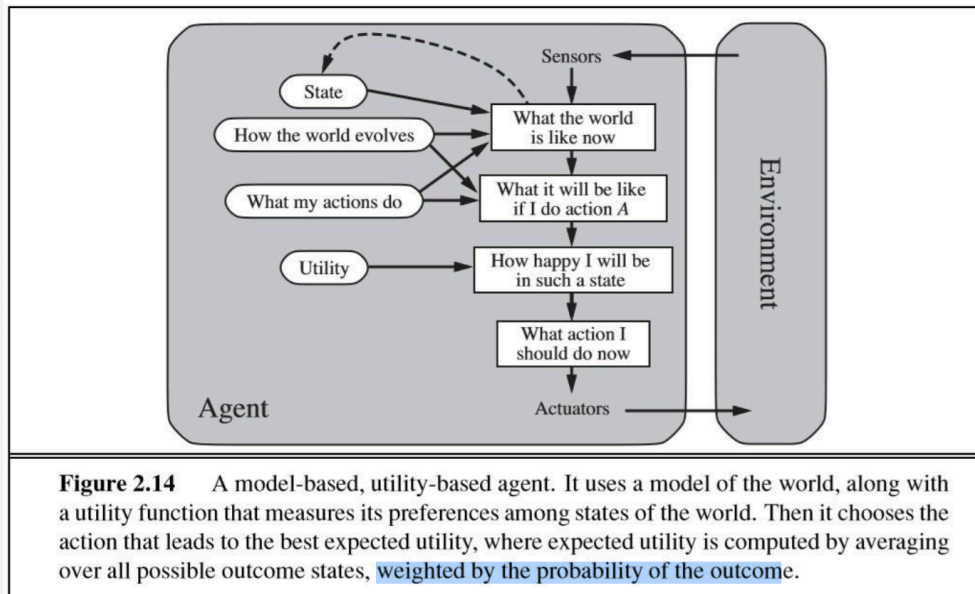
GOAL

I want to pass the exam

UTILITY ← optimized quality

I want to pass the exam with the highest grade

→ Utility based agents choose action that max. a numerical measure of desirability allowing to handle trade offs & uncertainty better than GBA.



Learning Agent

- ↳ AI tool capable of learning from its exp.
- ↳ starts with Basic Knowledge → Adapt Auto. through learning, to improve its own performance

Elements:

- 1) Learning → responsible for improving
- 2) Performance → selecting external actions
- 3) Critic → learning element uses feedback from Critic.
↳ how performance can be modified to do better in the future.
- 4) Problem Generator → suggesting action that will lead to new info. exp.

- When you were in school you would do a test and it would be marked the test is the critic. The teacher would mark the test and see what could be improved and instructs you how to do better next time.
- The **teacher** is the **LEARNING ELEMENT** and **you are the PERFORMANCE ELEMENT**.
- **PROBLEM GENERATOR**: - For example coming back to the school analogy, in science with your current knowledge at that time you would not have thought of placing a mass on a spring but the teacher suggested an experiment and you did it and this taught you more and **added to knowledge base**.

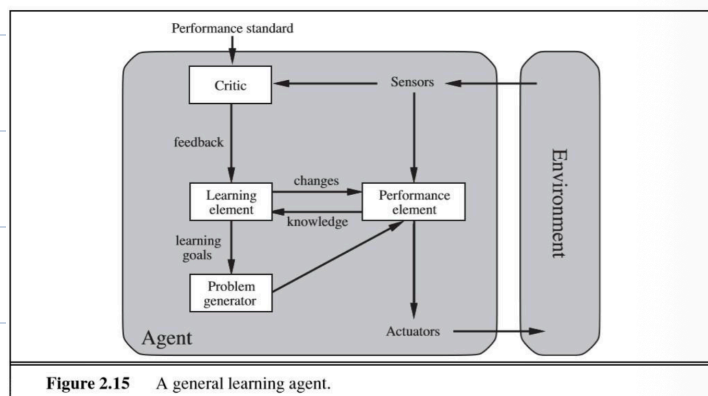


Figure 2.15 A general learning agent.

Learning in intelligent agents can be **summarized** as a process of modification of each component of the agent to bring the components into closer agreement with the available feedback information, thereby improving the overall performance of the agent.

Human is an example of a learning agent.

For example, a human can learn to ride a bicycle, even though, at birth, no human possesses this skill.